

DISTRIBUTED SYSTEMS, DISGUISED AS A FOLDER

How Cloud File Sync Actually Works

Why Dropbox, Google Drive, OneDrive and Box are much harder than they look — and a small, complete, working engine that shows exactly where the difficulty lives.

Companion code: the `syncbox/` package in this repository. About 2,900 lines of Python across twelve modules, standard library only, plus 600 lines of tests — 58 of them, including a randomised convergence fuzzer. A narrated demo runs with `python -m syncbox.demo` .

How to read this: each section names a problem, shows the mechanism that solves it, and points at the file where that mechanism lives.

A sync engine is not a file-copying program. It is a replicated database with an unusually hostile schema, an unusually unreliable network, and a user interface that promises none of that is happening.

1 Why this looks easy and isn't

Watch a folder. When a file changes, upload it. When the server has something new, download it. That is a weekend project, and it works beautifully right up until the moment there are two computers.

Everything hard follows from three facts that a single-machine mental model does not contain:

- **Changes originate in more than one place.** The server is not the author of the truth; it is one participant among several. Two machines can each make a change without knowing about the other's.
- **The network is not there when you need it.** Edits happen on planes, on trains, in basements. Work must accumulate locally and be delivered later, possibly much later, possibly in a different order than it was made.
- **Nothing can be undone.** A sync engine deletes and overwrites the user's real files, on the strength of something another machine told it. If the reasoning was wrong, the data is gone.

Put together, the requirement is not "copy files" but: *keep several independent replicas of a mutable tree converging toward the same state, while any of them may be editing, and any of them may be unreachable.*

That is a distributed systems problem with a folder icon on it.

2 The central insight: you need three views, not two

Here is the question that defeats the naive design. Your laptop has no file called `notes.txt`. The server has one. What should happen?

You cannot answer. Two completely different histories produce that exact situation:

History A

A friend created it on their machine.

LOCAL — absent —

REMOTE notes.txt

BASE — absent —

⇒ download it

History B

You deleted it here, last Tuesday.

LOCAL — absent —

REMOTE notes.txt

BASE notes.txt

⇒ delete it there

Identical LOCAL and REMOTE. Opposite correct actions. Only BASE distinguishes them.

BASE is what this machine and the server last agreed on. It is stored locally, it is updated only after a change has actually been applied, and it is the entire reason the engine can tell a *create* from a *delete* — or, more generally, tell *which side moved*.

THE ONE THING TO REMEMBER

Sync is a three-way merge, not a two-way comparison. LOCAL versus REMOTE tells you that the replicas differ. LOCAL versus BASE versus REMOTE tells you *why*, and only that answers what to do. This is the same structure git uses for a merge, for the same reason.

In the companion code this is `planner.py`, and it is deliberately a *pure function*: three dictionaries in, a list of actions out. No disk, no network, no clock. That is what makes it testable, and it is why the demo can print the plan and its reasoning before executing a single byte of it.

3 Identity: what makes two files "the same file"

Before you can compare three views you need to say what a file *is*. The tempting answers are all wrong.

Not the path. Files get renamed and moved, and the content did not change.

Not the modification time. Clocks disagree between machines, step backwards when NTP corrects them, and have coarse resolution on some filesystems. Worse, plenty of tools set mtime deliberately — `rsync -t`, tar extraction, build systems.

The content itself, named by a cryptographic hash. This one decision pays for itself repeatedly:

- Comparing versions across a network becomes a 16-byte comparison rather than a transfer.
- "Did this really change, or did something just touch the timestamp?" becomes answerable.
- Renames become free — same content id, different path, zero bytes transferred.
- Deduplication falls out automatically. Ten thousand users syncing the same PDF cost the provider one copy.

Chunking, and why a one-byte insert isn't a full re-upload

Hash the whole file and any change re-uploads everything. So split files into pieces and transfer only the changed pieces — but *not* at fixed offsets. Insert one byte at the front of a fixed-block file and every block shifts; nothing matches; you upload the lot.

Instead, let the content choose the boundaries. Slide a cheap rolling hash over the bytes and cut wherever its value hits a pattern. Now an edit perturbs only the chunks around it, and the boundaries re-synchronise on their own within one chunk. This is *content-defined chunking*, and it is the same trick underneath `rsync`, `restic`, `borg` and every serious backup tool.

Measured on the companion implementation, against a 300 KB file:

EDIT	CHUNKS REUSED	BYTES THAT CROSS THE WIRE
Insert 36 bytes in the middle	30 of 31	21,161 (7.1% of the file)
Overwrite 10 bytes near the end	30 of 31	12,839 (4.3%)
Prepend a header	30 of 31	58,281 (19.4%)

A fixed-block scheme would reuse nothing after the edit point in rows 1 and 3.

BUG FOUND WHILE WRITING THIS

The first version of the rolling hash built its 256-entry random table with `random.Random(seed).getrandbits(64)` inside the loop — re-seeding every iteration, so all 256 entries were identical. The hash then depended only on how many bytes it had seen, never on which bytes, and no cut condition ever fired. Chunking silently degraded to fixed maximum-size blocks and dedup did nothing. It still passed every "does the file round-trip correctly?" test. Measure the property you actually want.

4 Ordering: version vectors, because timestamps lie

Given two versions of a file, which is newer? If you answer with wall-clock timestamps, a laptop running forty seconds fast wins every race it enters, forever, including the ones it should lose.

But the deeper problem is that "newer" is the wrong question. The right one is: **did this edit happen knowing about that one?** Two people editing the same file on two planes produce two edits with different timestamps, and neither supersedes the other. Picking the later one silently destroys real work, with no error and no trace.

A *version vector* answers the right question. Each replica has a stable id; each file carries a map from replica id to a counter:

```
{"laptop": 3, "desktop": 1} # incorporates 3 edits from laptop, 1 from desktop
```

A replica bumps *only its own* counter when it changes a file. Comparison becomes structural rather than chronological:

RELATION	MEANING	WHAT THE ENGINE DOES
A dominates B	Every counter in A is $\geq$ B's, and they differ. A already contains everything B knows.	Safe to overwrite B with A
A == B	Same version.	Nothing
concurrent	A knows something B doesn't <i>and</i> B knows something A doesn't.	A genuine conflict. No algorithm can resolve it; a policy must.

That third row is the whole reason this problem is hard, and it is exactly the row timestamps paper over by pretending it cannot happen.

COST, HONESTLY

A version vector is $O(\text{number of replicas that ever touched the file})$. Fine for a handful of personal devices; a real problem at Drive scale with thousands of collaborators. Large systems use variants — server-assigned monotonic revisions with a single ordering point, dotted version vectors, or a "last writer plus causal context" set. The reasoning is identical; only the encoding shrinks.

5 What the server actually owes its clients

Surprisingly little, and all of it is about ordering.

1. **A total order on changes**, exposed as a resumable feed: "give me everything after position N." This is Dropbox's `/delta`, Drive's `changes.list`, OneDrive's `delta` query. The server being a single serialisation point is

what makes the distributed problem tractable at all.

2. **Compare-and-swap on every write.** The client says "store this, and I believe the current version is X." If it isn't X, the write is *rejected* — not merged, not clobbered.
3. **Idempotence.** Networks fail in the worst possible way: after the server commits, before the client hears about it. The client cannot distinguish that from "never arrived," so it must retry, and the retry must not duplicate.
4. **Integrity.** Verify the hash of everything stored and everything served.

Note what is *not* on the list: the server does not resolve conflicts. It detects them and refuses. Resolution happens on the client, because only the client can see both versions' content — and in an end-to-end-encrypted design the server could not read them even if it wanted to.

Why compare-and-swap, specifically

Without it you get the classic lost update. Two clients read version 5, both write version 6, and one edit vanishes with no error anywhere in the system. CAS converts that silent loss into a loud rejection, which the client then resolves with the ordinary three-way machinery. From the demo:

```
phone's plan, built against the world as it was:
push      notes/race.txt -- created locally

-- ...but the laptop commits first -----
push      notes/race.txt -- created locally

-- phone now executes its stale plan -----
note: server rejected our write (someone else committed first)
CAS rejections so far: 1

files surviving:
notes/race (conflicted copy from phone ...).txt -> "the phone's version"
notes/race.txt                                -> "the laptop's version"

neither edit was lost: True
```

Idempotence, and the deterministic operation id

The usual advice is "attach a unique request id." The subtlety is *where it comes from*. A randomly generated id is forgotten when the process restarts — which is precisely the moment you need it. So derive it from the intent instead:

```
def _op_id(replica, kind, path, content_id, vv):
    h = blake2b(digest_size=12)
    h.update("[".join([replica, kind, path, content_id, vv.to_json()]).encode())
    return f'[{replica}]:{h.hexdigest()}'
```

The same intent always produces the same id, across retries, across crashes, across restarts. The server recognises the replay and answers from its memo table rather than committing twice.

6 The decision table

All of the above collapses into one small table, evaluated per path. Let *local changed* mean LOCAL differs from BASE, and *remote changed* mean REMOTE's version vector carries something BASE's does not.

LOCAL CHANGED	REMOTE CHANGED	SITUATION	ACTION
no	no	In sync	Nothing
yes	no	We edited, created or deleted it	Push to server
no	yes	They edited, created or deleted it	Pull to disk
yes	yes	Both sides moved	Look at where they ended up ↓

WHERE BOTH SIDES ENDED UP	ACTION
Identical content	Not a conflict at all. Adopt the server's vector and move on. (Both machines ran the same formatter; both deleted the same junk file.) Catching this case prevents a great deal of spurious noise.
Deleted here, edited there	Resurrect. Keep the remote file. A deletion is cheap for a human to repeat; an edit is not.
Edited here, deleted there	Keep local and re-create it on the server. Same reasoning, other direction.
Both edited, and it's text	Attempt a three-way merge against the common ancestor.
Anything else	Conflict split: the remote version keeps the path, ours is preserved as a conflicted copy and uploaded as a file in its own right.

That last row hides a detail worth stating explicitly. *Why does the remote version keep the original name?* Because every peer resolving this same conflict independently then makes the same choice, and they converge without another round of negotiation. If the local version kept the name, two machines would each believe their own edit was canonical, and they would conflict again, forever.

Move detection

Once the per-path decisions exist, look at the plan as a whole. A delete of content X at one path plus a create of the same content X at another is almost certainly a rename. Collapsing that pair into a single move is what keeps renaming a 4 GB folder from becoming a 4 GB upload:

```
-- laptop renames archive/big.iso to vault/big.iso -----
move-remote  archive/big.iso -> vault/big.iso
      -- metadata only, no bytes transferred
      bytes uploaded for the rename: 0

-- phone applies the rename -----
move-local   archive/big.iso -> vault/big.iso
      -- renaming on disk instead of downloading
      bytes downloaded by phone: 0
```

It is a heuristic: copy a file and delete the original and the engine will call it a move, which produces the right final state anyway. Where the OS offers real file identity — inode on Unix, file id on NTFS — a production client tracks that instead of inferring.

7 Conflicts: there is no clever answer

If two people changed the same JPEG on two machines with no communication between them, no algorithm produces "the right" JPEG. The engineering question is not how to avoid that but what to do that never loses work and never surprises the user. Three families of answer:

Keep both — the conflicted copy

One version keeps the path; the other is renamed to something a human can understand: `budget (conflicted copy from laptop 2026-09-03 14-22-05).xlsx`. Impossible to lose data, requires no understanding of the file format, and the user can see what happened without reading a log. The vice is that it pushes the work onto the user, and in a shared folder it can spray copies everywhere. This is the universal default, and Dropbox's version is the one everyone recognises.

Merge, if you understand the format

Line-oriented text is the classic case. The three-way merge in the companion code is the one git uses: compare each side *against the common ancestor* rather than against each other, so an insertion is distinguishable from a deletion, and only genuinely overlapping regions are flagged.

```
ancestor: milk / bread / eggs
laptop:   milk / bread / eggs / coffee  # appended a line
phone:    oat milk / bread / eggs       # changed line 1

merged:   oat milk / bread / eggs / coffee # both edits kept, cleanly
```

Google Docs takes this much further: it does not store documents as files at all, but as an ordered stream of operations merged by operational transformation, so concurrent edits *compose* instead of conflicting. That is precisely why Docs has no conflicted copies while Drive-syncing a `.docx` does. The lesson generalises: **the**

conflict problem gets easier the more the system understands the data — and a general-purpose file sync engine, by definition, understands nothing.

A PRODUCT DECISION, NOT A TECHNICAL ONE

When a text merge *fails*, the companion engine does not write <<<<<< markers into the user's document. Conflict markers are a programmer's convention; a spreadsheet user opening a file full of them has simply had their file corrupted. Unmergeable means conflicted copy, like everything else.

Prevent conflicts entirely

Lock files while someone has them open (Box and SharePoint offer this), make everything append-only, or use CRDTs whose merge is mathematically guaranteed to converge. Each trades away either offline editing or generality, which is why none of them is the default for whole-filesystem sync.

8 Deletion, and the file that rises from the dead

The naive model is "a path either has a file or it doesn't." That model cannot express deletion in a distributed system, and the failure is famous.

Machine A deletes `notes.txt` while offline, then reconnects. The server sees that A no longer has the file and B still does. Is that a delete on A that must propagate to B, or a create on B that must propagate to A? **Absence carries no information.** Without a record saying "this was deliberately deleted, at this version," the deleted file comes back on the next sync — and comes back again, forever.

So deletion is not the removal of a record. It is the writing of a *different* record: a **tombstone**, with its own version vector, which propagates like any other change and can itself be superseded by someone re-creating the file.

```
-- laptop deletes it -----
delete-remote notes/temp.txt -- deleting locally; propagating
server record is now: notes/temp.txt [deleted {laptop:2}]

-- phone applies the deletion -----
delete-local  notes/temp.txt -- deleted elsewhere; removing our copy

phone still does not have it: True # after repeated re-syncs
```

Collecting tombstones without resurrecting anything

Tombstones accumulate, so they must eventually be collected — but you cannot expire them on a timer. A laptop that has been in a drawer for six months still has the file and still believes it is synced. If the tombstone is gone when it wakes up, the file looks brand new and gets helpfully re-uploaded.

So collection is gated on the **minimum cursor across all known replicas**: drop only what everybody has demonstrably seen. A replica that never returns pins its tombstones indefinitely — which is exactly why these products eventually unenroll devices that have been silent too long, and accept the (now explicit, now bounded) risk for those.

9 Touching the user's files without destroying them

This is where a bug is not a bug but a support ticket about a lost thesis.

Never write into the destination

```
write to a temporary file in the same filesystem
fsync the file           # bytes are actually on the medium
rename over the target   # atomic: readers see old or new, never half
fsync the directory     # the rename itself is durable
```

"Same filesystem" matters because a cross-device rename is really a copy plus a delete, which is not atomic at all. Both fsyncs matter too, and both are routinely skipped: without the first, the rename can be durable while the contents are not, and you get a correctly-named file full of zeros after a power cut. Without the second, the rename can be lost.

Deleting is never deleting

The engine deletes files because a *remote machine* said so. If anything upstream was wrong — a bug, a bad merge, a compromised account, a user who didn't mean it — that is unrecoverable and it is your fault. So local deletions move to a trash area and age out later. This is why every one of these products has a "restore deleted files" window: not as a courtesy, but because the engine can never be certain enough to actually unlink something.

Order of operations, and rename cycles

Actions must leave the tree in a legal state at every intermediate step: deletes deepest-first (you cannot remove a non-empty directory), creates shallowest-first (a parent must exist before its child), conflicts preserved before anything is overwritten.

And users really do swap two directories. A set of renames is a permutation, and every permutation decomposes into chains and cycles. Chains resolve themselves if you always pick a move whose destination nobody still has to vacate. Cycles have no such move — every destination is occupied by something itself waiting to move — and need exactly one temporary name to break:

```
def apply_moves(moves):
    drain_chains()           # any move whose destination is already free
    while pending:           # whatever is left must be a cycle
        origin, final = pending.popitem()
        move(origin, staging) # vacate one slot...
        drain_chains()       # ...and the cycle is now a chain
        deferred.append((staging, final))
    for staging, final in deferred:
        move(staging, final)
```

10 Being offline gracefully

Three mechanisms, and one idea underneath all of them.

Resumable uploads with no client-side progress state. On retry, ask the server which chunks it is still missing and send those. The server's own contents *are* the resume point, so there is no progress file to keep in sync, nothing to corrupt, and a resume works from a different machine. Content-addressed chunking and resumability turn out to be the same feature.

Exponential backoff with jitter. The jitter is not decoration. If a provider has an outage and ten million clients retry on identical schedules, recovery brings a synchronised thundering herd that knocks the service over again.

A write-ahead journal of intents, written before acting and cleared after, so a process killed mid-operation can replay. Combined with the ordering discipline:

```
apply the change → fsync → update BASE → clear the journal
```

Any other order and a crash in the gap either loses a change or re-applies one. Update BASE first and the client believes it synced something it did not — which is how a file silently stops syncing forever.

THE IDEA UNDERNEATH

Make every step re-runnable, and you no longer need to remember where you were. The reconciler derives its plan freshly from the world on every cycle, so there is no "resume from step 7" logic anywhere. This is the single biggest simplification available in the whole problem, and it is why the change feed can get away with cheap at-least-once delivery: applying a change you already have is a no-op, because the comparison is on content, not on events.

11 The mundane hell

None of this is intellectually interesting. All of it will generate bug reports.

- **Unicode normalisation.** macOS historically stored filenames decomposed (NFD): "é" as "e" plus a combining accent. Linux stores whatever bytes it is given. So `résumé.pdf` has two byte representations that are one name to a human. An engine that does not normalise will create both, forever, in a loop.
- **Case sensitivity.** Linux distinguishes `Report.txt` from `report.txt`; Windows and default macOS do not. Syncing a Linux folder containing both to a Mac is not a transfer problem, it is a modelling problem with no good answer.
- **Reserved names and characters.** `CON`, `PRN`, `NUL`, `COM1 – LPT9`, trailing spaces and dots, `: * ? " < > |` — all fine on Linux, all impossible or silently mangled on Windows. A Linux client can otherwise upload a file that no Windows peer can ever materialise.
- **Path length.** Classic Windows caps at 260 characters, so a deeply nested tree that syncs fine everywhere else fails on one machine — and fails *partway*, leaving that replica permanently unable to converge.
- **Symlinks, hardlinks, sparse files, extended attributes, resource forks, permissions.** Follow a symlink and you may loop forever or export files from outside the sync root. Store it faithfully and Windows cannot represent it. Whatever you choose, choose it explicitly.
- **Watcher overflow.** inotify queues fill up under load and tell you only that events were lost. FSEvents coalesces. Nothing at all fires while your process is not running, and network mounts often have no watcher. So the file watcher is an *optimisation* that tells you where to look, and a periodic full scan remains the correctness backstop. You can never delete the scanner.
- **The hash cache is a lie you accept.** Rehashing 100 GB every cycle is out of the question, so clients cache `(inode, size, mtime, ctime) → hash`. It is possible to modify a file and land on the same size and mtime, so the mitigation is to include ctime (which a user cannot set directly), re-hash anything modified within a timestamp tick of now, and re-verify everything in the background periodically.
- **Files that change while you read them.** Databases, virtual machine images and log files are written continuously. You will upload a torn read; the question is only whether you notice.

12 What you are actually promising

State the consistency model plainly, because users will assume something much stronger:

- **Per file: linearizable.** Every version of a given path is totally ordered by the compare-and-swap chain, and no update is ever silently lost.
- **Across files: not atomic.** There is no transaction spanning two paths. Save a `.tex` and its `.bib` together and a peer may briefly see the new one with the old other. Every consumer product works this way, because cross-

file transactions over a wide area network with offline participants would mean blocking on unavailable peers.

- **Overall: eventually consistent.** If everyone stops editing and stays connected long enough, all replicas converge to the same state. Not immediately, not atomically. *Eventually*.

Convergence is therefore the property worth testing, and it is the one the companion code fuzzes: three replicas, random creates, edits, deletes and renames, random disconnections, 30% packet loss, then everybody reconnects and syncs until quiet. The assertion is not *what* the final tree contains — that depends on conflict policy — only that all three replicas agree on it.

CASE STUDY: THE BUG ONLY THE FUZZER COULD FIND

At one seed in forty, three replicas failed to converge. The cause was in move detection. Replica *three* had deleted `b.txt` locally and happened to hold a file elsewhere with `b.txt`'s old content, so the planner paired them into a rename. But the destination path already had its own history on the server — a tombstone written by the other two replicas. The synthesised move carried the *source* path's version vector to the *destination*, where it could not dominate what was already there, so compare-and-swap rejected it. The next cycle rescanned, reasoned identically, and produced exactly the same doomed plan.

Not a crash, not a transient failure: a permanent stall, invisible except as one replica quietly never catching up. The fix is a one-line guard — only collapse a delete and a create into a move when the destination is unknown to the server as well as to us — but no amount of staring at the decision table would have surfaced it. **Interleaving bugs are found by interleaving.**

13 What a real one adds

The companion engine is complete in structure and deliberately small in scope. The distance to a product is mostly in these:

- **Scale of metadata.** Hundreds of millions of users' file metadata does not fit in one database. Shard by user or by namespace, and accept that a shared folder spanning shards is now a distributed transaction problem of its own.
- **Sharing and permissions.** This is where the difficulty genuinely jumps. Other people's ACL changes become events in your feed; a file can leave your namespace because someone else revoked access; two users with different permissions must see consistent but different trees.
- **Streaming.** Files larger than memory need chunk-at-a-time hashing, upload and verification rather than the whole-blob approach used here for clarity.
- **Concurrency.** Real clients run many transfers at once with a bounded pool, while preserving per-path ordering.
- **Selective sync and placeholders.** "This file exists but isn't downloaded" means a filesystem driver — Windows Cloud Files API, macOS File Provider — and a whole second consistency problem between the placeholder state and the real state.

- **Encryption.** End-to-end encryption breaks server-side dedup and makes conflict resolution strictly a client concern. Convergent encryption gets some dedup back and leaks whether you have a given file.
- **LAN sync,** bandwidth shaping, battery and thermal awareness, and the enormous amount of work in *not* syncing at the wrong moment.

14 Running the companion code

```
python -m syncbox.demo           # 11 narrated scenarios
python -m unittest discover -s tests -t .    # 58 tests, ~3 seconds
```

FILE	WHAT IT IS FOR
planner.py	The reconciler. Pure function, three views in, an ordered plan out. Start here.
clock.py	Version vectors: newer vs. different vs. concurrent.
hashing.py	Content addressing and content-defined chunking.
fsmodel.py	Nodes, tombstones, path normalisation and safety.
scanner.py	Cheap snapshots via the hash cache; the watcher shim.
db.py	The client's durable memory: BASE, the REMOTE mirror, the journal.
server.py	Blob store, compare-and-swap metadata, change feed, tombstone GC.
transport.py	A deliberately unreliable network, with resumable uploads.
applier.py	Atomic writes, trash, rename cycles.
merge.py	Conflict policy: conflicted copies and three-way text merge.
client.py	One sync cycle, assembled from the above.
demo.py	The scenarios, narrated with the planner's own reasoning.

15 Where to read further

- **Unison** (Benjamin Pierce et al.) — the academic bidirectional synchroniser. Its papers state the reconciliation algorithm and its correctness properties precisely; this is the clearest formal treatment of the three-way structure.
- **The rsync algorithm** (Tridgell & Mackerras, 1996) — the origin of rolling-hash delta transfer.
- **"Dynamo: Amazon's Highly Available Key-value Store"** — version vectors, concurrent writes and application-level conflict resolution, in a system that made those trade-offs explicit.
- **"A comprehensive study of Convergent and Commutative Replicated Data Types"** (Shapiro et al.) — CRDTs, i.e. what becomes possible when merge is guaranteed to converge by construction.

- **Dropbox's engineering blog on Nucleus** — a rewrite of the sync engine in Rust, and unusually candid about which parts of the original design did not survive contact with scale.
- **Git's merge internals** — the same three-way merge, very well documented, in a system you can inspect at will.

Written as a companion to the `syncbox` package in this repository. The measurements, traces and the case study in section 12 are real output from that code, not illustrations.